

Güvenli Programlamaya Giriş ve Uygulama Koruma Planı

CEN429 Güvenli Programlama — Hafta 1

Dr. Öğr. Üyesi Uğur CORUH · 18.09.2026

0. Temel kavramlar (sıfırdan)

Neden bu bölüm?

Bu ders "güvenlik", "zafiyet", "tehdit modeli" gibi terimlerle dolu.

Hiçbirini bilmediğinizi varsayıyoruz.

Önce hepsini **tek tek** tanımlayalım.

Güvenlik nedir?

- **Güvenlik:** bir sistemi, ona **zarar vermek isteyen** birine karşı korumak.
- Normal hata: yanlışlıkla olur.
- Güvenlik: **kasıtlı** bir saldırgan var.

Varlık (asset) nedir?

- **Varlık:** korumaya değer her şey (veri, anahtar, işlem).
- Örnek: parola, kredi kartı, lisans, kullanıcı verisi.
- Güvenlik "neyi koruyoruz?" ile başlar.

Tehdit ve zafiyet

- **Tehdit:** olabilecek kötü bir olay (veri çalınması).
- **Zafiyet:** bunu mümkün kılan **zayıf nokta** (denetimsiz girdi).
- Tehdit + zafiyet + saldırgan = risk.

CIA üçlüsü

Güvenliğin üç temel hedefi:

- **Gizlilik (C):** yalnız yetkili görsün.
- **Bütünlük (I):** izinsiz değişmesin.
- **Erişilebilirlik (A):** gerektiğinde çalışsın.

Saldırgan modeli

- **Saldırgan modeli:** "saldırgan neyi görebilir/yapabilir?"
- Ağdan mı bağlanıyor, cihaza mı sahip?
- Savunmayı buna göre tasarlarız.

Beyaz kutu / MATE

- **MATE (Man-At-The-End):** programa **sahip** saldırgan.
- Kodu okur, belleği görür, değiştirir.
- Mobil/masaüstü uygulamanın gerçek durumu (bu dersin ana teması).

Tehdit modelleme

- **Tehdit modelleme:** "neyi, kime karşı, nasıl koruyacağız?" sorusunu **sistemli** yanıtlamak.
- Varlıkları, tehditleri, önlemleri listeler.
- Bugün STRIDE ve saldırı ağaçlarıyla yapacağız.

STRIDE

- Tehditleri altı harfle sınıflandıran yöntem:
- **S**poofing, **T**ampering, **R**epudiation, **I**nformation disclosure, **D**enial of service, **E**levation of privilege.
- Her öge için "hangi tehdit?" diye sorar.

Saldırı ağacı

- **Saldırı ağacı:** bir hedefi (ör. "anahtarı çal") **alt adımlara** bölen ağaç.
- Kök: saldırganın amacı.
- Dallar: bunu başarmanın yolları.

Veri akış diyagramı (DFD)

- **DFD:** verinin sistemde **nasıl aktığını** gösteren şema.
- Süreç, veri deposu, dış varlık, akış, güven sınırı.
- Tehditleri bulmak için harita.

Katmanlı savunma

- **Katmanlı savunma (defense in depth):** tek önlem değil, **birçok** önlem.
- Biri aşılsa diğeri durdurur.
- "Güvenlik kabuğu" bu katmanların birleşimi.

Güvenli tasarım ilkeleri

- Saltzer & Schroeder (1975): en az yetki, güvenli varsayılan, ekonomi, açık tasarım...
- Bugün hâlâ geçerli.
- Bu dersin omurgası.

Ödünleşim (trade-off)

- Her koruma bir **bedel** getirir: hız, karmaşıklık, maliyet.
- Güvenlik "sonsuz koruma" değil, **dengeli** koruma.
- Kalan riski açıkça yazarız.

Şimdi hazırız

Terimler:

güvenlik · varlık · tehdit/zafiyet · CIA · saldırgan modeli · MATE · tehdit modelleme · STRIDE · saldırı ağacı · DFD · katmanlı savunma · tasarım ilkeleri · ödünleşim

Şimdi: güvenlik nedir, derinlemesine.

Bugünün planı (3 saat)

Saat	Konu
1	Ders tanıtımı · Güvenlik nedir? · Saldırgan · İlkeler · Uygulama korumasına genel bakış
2	Koruma planı · STRIDE ve risk puanı · Saldırı ağacı · Uygulamalı örnek "Kasa" · Sınıf alıştırmaları
3	Demo 1–2 · Güvenli başlatma · Taşmalar · Demo 3–4 · Bellek yönetimi · Bölümleme · Güvenli süreç · Proje

Demolar: `code/week-01` — Windows `.\demo.ps1` · WSL / Linux `sh demo.sh` · Visual Studio: Klasör
Aç → `code`

Ders nasıl yürüyor?

- **Tek dönem projesi:** C/C++ uygulaması + "sertifikasyondan geçecekmiş gibi" bir **güvenlik kılavuzu**
 - Vize kontrolü: **7. hafta** gösterim · Final kontrolü: **15. hafta** gösterim
- **İki quiz:** 8. hafta (1–6) · 16. hafta (9–14)
- Not: $Vize = 0,6 \cdot Projel + 0,4 \cdot Quiz1$ · $Final = 0,7 \cdot Proje2 + 0,3 \cdot Quiz2$
- Her hafta: **hatalı kod** → **saldırı** → **düzeltilme**



Etik: Teknikleri yalnız **kendi bilgisayarınızda** ve verilen demolar üzerinde deneyin.

1. Güvenlik nedir?

Kasa benzetmesi

Kavram	Kasa	Yazılım
Varlık	Para	Parola, ödeme anahtarı
Tehdit	Hırsız	Parolayı çalmak isteyen
Zafiyet	Arızalı kilit	Denetlenmeyen <code>strcpy</code>
Saldırı	Kilidi açmak	Uzun adla yönetici olmak
Karşı önlem	Yeni kilit	Uzunluk denetimi
Risk	Olasılık × etki	"Bulunursa bütün hesaplar gider"

Güvenliğin üç hedefi: CIA

	Soru	Bozulursa
C onfidentiality — Gizlilik	Yalnız yetkili görebiliyor mu?	Parolalar sızar
I ntegrity — Bütünlük	Yetkisiz değiştirilebiliyor mu?	Bakiye, not değişir
A vailability — Erişilebilirlik	Gerektiğinde çalışıyor mu?	Sistem çöker

+ Kimlik doğrulama · Yetkilendirme · İnkâr edilemezlik

Soru: Bankacılık uygulaması bakiyeyi şifresiz yazıyor → ? · Alıcı IBAN yolda değişiyor → ? · Bayram gecesi çöküyor → ?

CIA üçlüsü — şema



Sorunun cevabı: bakiye şifresiz → **C**; IBAN yolda değişiyor → **I**; bayram gecesi çöküyor → **A**.

Hata ≠ zafiyet, ama...

Güvenlik açıklarının çok büyük kısmı **sıradan yazılım hatalarıdır**.

Heartbleed (2014, CVE-2014-0160)

- İstemci: "Bana **64 KB** geri yolla" — ama yalnız **birkaç bayt** gönderir
- Sunucu uzunluğu **denetlemez** → komşu belleği (parolalar, anahtarlar) geri yollar
- Tek bir eksik uzunluk denetimi → internetin önemli bir kısmı etkilendi

Saldırgan kim, neye erişebiliyor?

Saldırgan	Erişim	Savunma
Uzaktaki	Yalnız ağ mesajları	Girdi doğrulama, TLS
Yerel kullanıcı	Dosyalar, ortam değişkenleri	İzinler, güvenli başlatma
İçeriden kişi	Kaynak kod, sunucu	Ayrıcalık ayrımı, denetim kaydı
Cihazın sahibi (beyaz kutu)	Bellek, hata ayıklayıcı, ikili kod	RASP, kod gizleme, whitebox

Bu dersin farkı: program **saldırganın cihazında** çalışıyorsa ne yaparız?

Güvenli tasarım ilkeleri (Saltzer & Schroeder, 1975)

İlke	Tek cümle
En az ayrıcalık	Yalnız gereken yetkiyle çalış
Güvenli varsayılan	Varsayılan "izin yok"
Tam aracılık	Her erişimi her seferinde denetle
Açık tasarım	Güvenlik algoritmanın değil anahtarın gizliliğine dayanır
Mekanizma ekonomisi	Basit = daha az hata
Ayrıcalıkların ayrılması	Kritik işe iki koşul
En az ortak mekanizma	Paylaşılan kaynağı azalt
Kabul edilebilirlik	Kullanılamayan güvenlik atlatılır

+ **Derinlemesine savunma**: tek önleme güvenme, **katman** kur.

2. Uygulama korumasına genel bakış

Neden tek bir önlem yetmez?

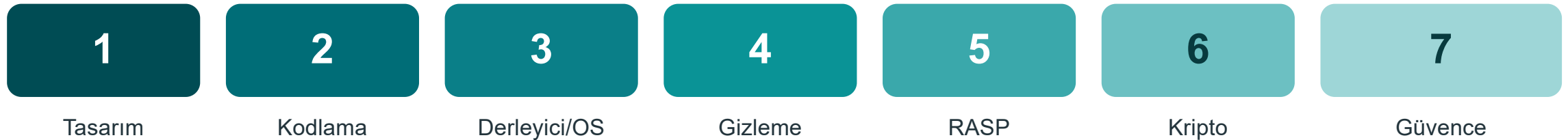
Önlem	Neyi çözmez?
Hatasız kod	Saldırgan ikili dosyayı açıp mantığı okur
Gizlenmiş kod	Hata ayıklayıcıyla durdurup belleği okur
İzlenmeye karşı korunan bellek	Anahtar kodda düz metin → dize taraması yeter
Gizlenmiş anahtar	Tek bir denetimi değiştirip korumayı atlar

→ Her katman bir öncekinin açığını kapatır; saldırgan hepsini **aynı anda** aşmalı

Yedi katman: içten dışa

#	Katman	Soru	Hafta
1	Güvenli tasarım	Neyi, kime karşı?	1–2
2	Güvenli kodlama	Kodum hatasız mı?	1, 4, 5
3	Derleyici/OS korumaları	Hata kaçarsa sömürü zor mu?	4
4	Gizleme	Kodu okuyan anlayabiliyor mu?	4, 5, 9, 14
5	RASP	Saldırı altında olduğunu fark ediyor mu?	6
6	Kriptografi	Veri ele geçse bile anlamsız mı?	3, 10, 11
7	Güvence	Çalıştığını nasıl kanıtlıyoruz?	12–13

Yedi katman — şema (içten dışa)



içeride: "doğru yaz"

dışarıda: "kod saldırganın elindeyken"

Bir hata dış katmanları geçse bile **bir sonraki** katman durdurmayı dener (derinlemesine savunma).

Güvenlik kabuğu: katmanların birleştiği yer

Hassas bir anahtar **aynı anda** birkaç katmanın içinde taşınır:

1. Diskte **şifreli** durur
2. Yalnız kullanılacağı an, yalnız gerektiği kadar **çözülür**
3. Çözüldüğü kod bölümü **gizlenmiş** ve **bütünlüğü denetlenmiş**
4. Süreçte **hata ayıklayıcı denetimi** çalışıyor

➔ 3. haftada dört kabuklu bir tasarımı adım adım kuracağız

Hangi katman ne zaman? Saldırgan modeline göre

Uygulama	Saldırgan nerede?	Öncelik
Sunucu web hizmeti	Ağda; kod ve sunucuya erişimi yok	1, 2, 3, 6, 7
Masaüstü iş uygulaması	Kullanıcı bilgisayarında; kullanıcı saldırgan değil	1, 2, 3, 6
Mobil ödeme	Cihaz saldırganın elinde olabilir	Hepsi; özellikle 4, 5, 6
Oyun, lisanslı yazılım	Kullanıcı saldırgan	4, 5
Gömülü cihaz	Fiziksel erişim	2, 3, 6 + donanım

Beyaz kutu saldırgan: programın kendisine sahip — kopyalar, açar, değiştirir, tekrar tekrar çalıştırır



Gizleme doğru kodun yerine geçmez

- Gizleme saldırganı **yavaşlatır**, hatayı **düzeltilmez**
- Gizlenmiş bir taşma hâlâ bir taşmadır
- Dış katmanlar ancak iç katmanlar sağlamsa anlam kazanır

Sunucu, istemciye asla güvenmez. Gizlenmiş istemci bile sunucuya tutarsız veri gönderebilir.

Korumanın bedeli

Bedel	Örnek
Performans	Kontrol akışı düzleştirme bir fonksiyonu birkaç kat yavaşlatabilir
Boyut	Sanallaştırma, whitebox tabloları ikili dosyayı büyütür
Bakım	Gizlenmiş kodun çökme raporu okunmaz
Yanlış pozitif	Aşırı hassas root algılama meşru kullanıcıyı cezalandırır

→ Her önlem **gerekçesiyle** yazılır: *şu varlığı, şu saldırıgana karşı, şu maliyetle*

Değerlendirici tabloyu tersten okur

1. Dize taraması → anahtar, URL, hata iletisi görünüyor mu?
2. İkili dosyayı açma → mantık okunuyor mu?
3. Hata ayıklayıcı bağlama → program fark ediyor mu?
4. Kodu değiştirme → bütünlük denetimi yakalıyor mu?
5. Bellek dökümü → sır bulunuyor mu?

Durdurulmadığı **her adım** raporda bir bulgu olur

Kendini dene (3 dk)

1. **Çevrimdışı parola yöneticisi** için saldırgan modeli nedir? Hangi katmanlar önemli?
2. "Sunucuda çalışıyor, gizlemeye gerek yok" — ne zaman doğru, ne zaman yanlış?
3. Yalnız gizleme kullanan bir oyunda hangi yol açık kalır?

Kendini dene — cevaplar

1. **Çevrimdışı parola yöneticisi**: saldırgan cihaza/dosyaya sahip (beyaz kutu). Önemli katmanlar: **kriptografi** (paroladan KDF + AEAD), **güvenli kodlama**, bir miktar gizleme/RASP.
2. **"Sunucuda çalışıyor"**: istemciye hassas mantık/anahtar **gönderilmiyorsa doğru**; istemcide çalışan kod/anahtar varsa **yanlış** (beyaz kutu).
3. **Yalnız gizleme**: sunucu tarafı doğrulama yoksa, istemci hilesi (yamalı istemci) açık kalır — gizleme mantığı **düzelmez**.

3. Uygulama koruma planı ve tehdit modelleme

Uygulama koruma planı: 7 adım

1. **Kapsam** — Neyi koruyoruz? Hangi ikili, hangi sürüm, hangi özet?
2. **Mimari ve arayüzler** — Kim kiminle, hangi kanaldan konuşuyor?
3. **Varlıklar** — Nerede durur, ne zaman silinir, **C** mi **I** mi?
4. **Tehditler** — Saldırgan kim, varlığa hangi yoldan ulaşır?
5. **Karşı önlemler** — Hangi katman, hangi önlem?
6. **Doğrulama** — Çalıştığı nasıl kanıtlanıyor?
7. **Kalan risk** — Kabul edilenler ve nedenleri

→ Sertifikasyonda bu plan **yüzlerce sayfalık bir güvenlik kılavuzudur**. Siz projede **20–30 sayfalık** sürümünü yazacaksınız.

Örnek mimari: mobil ödeme uygulaması

TELEFON (güvenilmez)

SUNUCU (güvenilir)

Mobil uygulama

| A

Kütüphane - Java — D: TLS + mesaj şifreleme —> Arka uç + HSM

| B (JNI)

Kütüphane - native C/C++ — E: NFC —> Ödeme terminali

| C

Yerel veritabanı (şifreli)

- Telefon **saldırganın elinde** olabilir: root, hata ayıklayıcı, bellek dökümü
- En hassas işler **native C/C++** katmanında

Arayüz ve varlık tabloları

Arayüz	Uçlar	Kimlik doğrulama	Gizlilik / bütünlük
C	native – veritabanı	—	Şifreleme + MAC, anahtar cihaza bağlı
D	SDK – sunucu	Sertifika sabitleme	TLS + ayrıca mesaj şifreleme

Varlık	Nerede	Ömür	Koruma
Tek kullanımlık ödeme anahtarı	DB (şifreli), bellek	Tek ödeme	C, I
Oturum anahtarı	Yalnız bellek	Oturum	C, I
İşlem sayacı	DB	Kalıcı	I

Listede olmayan varlık korunmuyor demektir.

STRIDE

	Tehdit	Bozduğu	Örnek
S	Kimlik taklidi	Kimlik doğrulama	Sahte uygulama SDK'yı çağırır
T	Kurcalama	Bütünlük	Native denetim ikili dosyada değiştirilir
R	İnkâr	İnkâr edilemezlik	"Bu ödemeyi ben yapmadım"
I	Bilgi sızması	Gizlilik	Anahtar bellek dökümünden okunur
D	Hizmet engelleme	Erişilebilirlik	Binlerce sahte kayıt isteği
E	Yetki yükseltme	Yetkilendirme	Sıradan kullanıcı → yönetici

Veri akış diyagramını çiz → **güven sınırını geçen her ok** için altı harfi sor.

Saldırı ağacı: "Ödeme anahtarını ele geçir"

HEDEF: Ödeme anahtarı — *VEYA* (biri yeter)

1. **Veritabanını çöz** — *VE* (hepsi gerekir): root yetkisi al + veritabanı anahtarını bul
2. **Bellekten oku** — *VEYA*: hata ayıklayıcı bağla | bellek dökümü al | fonksiyona kanca at
3. **Trafiği dinle** — *VE*: TLS'i kır + mesaj düzeyi şifrelemeyi kır

Saldırı ağacından çıkan sonuçlar

- "Trafiği dinle" yolu **iki katmanı birden** kırmayı gerektirir → pahalı → **derinlemesine savunma** çalışıyor
- "Bellekten oku" yolunun **üç alternatifi** var → en zayıf nokta **bellekteki anahtar**
 - Bu hafta: **Demo 2** (bellekte kalan sır)
 - Hafta 6: çalışma zamanı koruması (hata ayıklayıcı, kanca algılama)
 - Hafta 11: whitebox kriptografi (anahtar bellekte hiç açık durmaz)
- Savunmacının hedefi: saldırganı **VE** düğümlerine zorlamak

Veri akış diyagramı: beş sembol

Sembol	Adı	Örnek
□ Dikdörtgen	Dış varlık	Kullanıcı, ödeme terminali
○ Daire	Süreç	Giriş modülü, kript kütüphanesi
= İki çizgi	Veri deposu	Veritabanı, yapılandırma dosyası
→ Ok	Veri akışı	HTTP isteği, fonksiyon çağrısı
--- Kesikli çizgi	Güven sınırı	İnternet ↔ sunucu, uygulama ↔ OS

→ Güven sınırını geçen her oku **kırmızıya** boyayın: incelemeye oradan başlayın

Hangi öğeye hangi harf?

Öge	S	T	R	I	D	E
Dış varlık	✓		✓			
Süreç	✓	✓	✓	✓	✓	✓
Veri deposu		✓	✓ ¹	✓	✓	
Veri akışı		✓		✓	✓	

¹ Denetim kaydıysa · Veri akışının "kimliği" yoktur → **S sorulmaz**

Her harf için soru ve önlem

Harf	Kendinize sorun	Tipik önlem
S	Karşı taraf gerçekten o mu?	Kimlik doğrulama, imza, sertifika sabitleme
T	Değiştirilse fark eder miyim?	MAC/HMAC, imza, bütünlük denetimi
R	"Ben yapmadım" derse kanıtım var mı?	Kurcalamaya dayanıklı günlük, imzalı işlem
I	Diskte, ağda, bellekte, günlükte sızar mı?	Şifreleme, bellek temizleme, maskeleye
D	Çökertilebilir, tüketilebilir mi?	Girdi sınırı, oran sınırlama, kota
E	Az yetkili, çok yetkilinin işini yaptırabilir mi?	En az ayrıcalık, tam aracılık, bellek güvenliği

Bir taşma = **D** (çöker) + **E** (akış ele geçer) + **I** (bitişik sır okunur)

Risk puanı: olasılık $\times$ etki

	Etki 1	Etki 2	Etki 3
Olasılık 3	3	6	9 kritik
Olasılık 2	2	4	6
Olasılık 1	1	2	3

- **Olasılık:** fiziksel erişim mi, internetten mi? Uzmanlık? Otomatikleşir mi?
- **Etki:** hangi varlık, kaç kullanıcı, parasal/hukuki sonuç?
- Daha ayrıntılı: **CVSS** (2. hafta), **saldırı potansiyeli** (13. hafta)

Tehdide dört yanıt

1. **Azalt** — önlem ekle (en sık)
 2. **Ortadan kaldır** — özelliği hiç yazma (en güçlü)
"Parolayı hatırla" yoksa, diskteki parola tehdidi de yok
 3. **Aktar** — kart verisini saklama, ödeme sağlayıcısına bırak
 4. **Kabul et** — gerekçesiyle **kalan risk** listesine yaz
- ⚠ Plana yazılmamış risk kabul edilmiş değil, **gözden kaçmıştır**

4. Uygulamalı örnek: "Kasa" parola kasası

Adım 0–1: Uygulama ve kapsam

Kasa: C++ masaüstü parola kasası

- Ana parola → türetilen anahtar → kayıtlar tek dosyada şifreli
- Panoya kopyalama · isteğe bağlı bulut yedek · otomatik güncelleme

Alan	Değer
Değerlendirme hedefi	kasa 1.0 + kasa_cekirdek + yapılandırma
Kapsam dışı	Yedek sunucusunun kendisi, işletim sistemi
Güvenlik amacı	Ana parolayı bilmeyen için okunamaz ve fark edilmeden değiştirilemez

Adım 2: Mimari ve arayüzler

Arayüz	Uçlar	Kimlik doğrulama	Gizlilik / bütünlük
A	Kullanıcı → Arayüz	Ana parola	Ekranda gösterilmez
B	Arayüz → Çekirdek	Aynı süreç	Parola çağrı sonrası silinir
C	Çekirdek → Kasa dosyası	—	AES-GCM, anahtar paroladan
D	Çekirdek → Pano	—	30 s sonra temizlenir
E	Çekirdek → Yedek sunucu	Sertifika + belirteç	TLS + zaten şifreli dosya
F	Arayüz → Güncelleme	Paket imzası	TLS; asıl güvence imza

Adım 3: Varlıklar

#	Varlık	Nerede	Oluşma → silinme	Koruma
V1	Ana parola	Yalnız bellek	Yazılır → türetilir → hemen silinir	C
V2	Kasa anahtarı	Yalnız bellek	Türetilir → kilitlenince silinir	C, I
V3	Çözülmüş kayıt	Bellek	Görüntülenir → kapanınca silinir	C
V4	Kasa dosyası	Disk, yedek	Kalıcı	C, I
V5	Tuz, türetme parametresi	Dosya başlığı	Oluşturulurken	I
V6	Güncelleme açık anahtarı	Uygulamaya gömülü	Derlemede	I

Koruma sınıfları: **C** gizlilik · **I** bütünlük · **I+** bütünlük + hesap verebilirlik · **N** yok

Adım 4: Tehditler

#	Nerede	Harf	Tehdit	Risk
T1	C / V4	I	Çalınan dosya, zayıf parola çevrimdışı denenir	9
T2	B / V1-2	I	Parola/anahtar bellekte kalır → döküm, takas	6
T3	D / V3	I	Panodaki parolayı başka uygulama okur	6
T5	C / V5	T	Yineleme sayısı 1'e düşürülür	3
T6	F / V6	T, E	Sahte güncelleme kurulur	6
T8	Süreç	T	Kasa ayrıştırıcısında bellek hatası	6

Adım 5: Karşı önlemler

Tehdit	Önlem	Katman
T1	Yavaş türetme (Argon2id / yüksek yinelemeli PBKDF2) + parola gücü şartı	6, 2
T2	Güvenli silme, mlock , döküm kapatma	2
T3	Pano 30 s sonra temizlenir	1
T4-5	Kimlik doğrulamalı şifreleme; başlık = AAD ; en düşük yineleme kodda sabit	6, 2
T6	İmzayı doğrulamadan asla çalıştırma; geri almayı reddet	6
T8	Uzunluk doğrulama + fuzzing + derleyici korumaları	2, 3, 7

Adım 6–7: Doğrulama ve kalan risk

Önlem	Test	Beklenen
Türetme	Süre ölçümü	≥ 250 ms
Silme	Kilitten sonra bellek dökümünde parola ara	Bulunmaz
Bütünlük	Dosyanın bir baytını değiştir	"Bozuk" der, açmaz, çökmez
İmza	Bozuk imzalı paket	Reddedilir, günlüğe yazılır

Kalan risk: kasa açıkken aynı kullanıcıyla çalışan zararlı · omuz sörfü · çok zayıf parola

⚠ Testi olmayan önlem değerlendirici gözünde **yoktur**

Sık yapılan hatalar

- **Varlık unutmak:** anahtarlar, belirteçler, sayaçlar, yapılandırma, **kodun kendisi**
- **Güven sınırını yanlış çizmek:** kendi istemcinizi "güvenilir" saymak
- **Önlemi tehditle eşleştirmemek:** "TLS kullanıyoruz" — hangi tehdidi kapatıyor, hangisini kapatmıyor?
- **Testsiz önlem:** "siliyoruz" deyip derleyicinin silmeyi kaldırdığını fark etmemek (Demo 2)

Sınıf alıştırması (15 dk)

Öğrenci not sistemi: öğretim üyesi not girer, öğrenci görür, veri sunucuda.

1. Veri akış diyagramı + güven sınırları
2. Her STRIDE harfinden **bir** tehdit
3. "Notumu 45'ten 85'e çıkar" saldırı ağacı — **en ucuz yol** hangisi?

5. Program başlarken ve bellekte kalan sırlar

Program boş sayfayla başlamaz

Çalıştıran kişiden **devraldıkları**:

- Ortam değişkenleri: `PATH`, `LD_PRELOAD`, `IFS` ...
- Açık dosya tanıtıcıları, çalışma dizini, `umask`
- Kaynak sınırları, çökme dökümü ayarı

Program bunlara güvenirse, **onları kontrol eden programı kontrol eder.**

Kitap: Viega & Messier, Tarif 1.1–1.9

Demo 1 — PATH ile kandırma (CWE-426)

```
int durum = system(KOMUT); /* Linux: "date"  Windows: "hostname" */  
                          /* HANGİSİ çalışacak? */
```

`.\demo.ps1` (Windows) · `sh demo.sh` (WSL / Linux)

```
ADIM 2 – PATH="$PWD/sahte:$PATH" ./bin/linux/rapor
```

```
Rapor tarihi:
```

```
!!! SAHTE 'date' calisti !!!
```

```
ADIM 3 – ./bin/linux/rapor_guvenli -> Rapor tarihi: Sat Sep 19 ...
```

Düzeltilme: üç kural

```
char *const arguman[] = { "date", NULL };  
char *const temiz_ortam[] = { "PATH=/usr/bin:/bin", "LANG=C", NULL };  
posix_spawn(&pid, "/bin/date", NULL, NULL, arguman, temiz_ortam);
```

1. **Mutlak yol** → PATH araması yok
2. **Kabuk yok** (`posix_spawn` / `execve`) → özel karakter, `IFS` yok
3. **Temiz ortam** → bilinen, küçük bir ortam

Windows: `CreateProcessW` + `System32\hostname.exe` + yalnız `SystemRoot`

En iyisi: başka programa hiç gerek bırakma → `strftime()`, `GetComputerNameW()`

Güvenli başlatma: ne devralınıyor?

Devralınan	Tehlike	Tarif
Ortam değişkenleri	<code>PATH</code> , <code>LD_PRELOAD</code> yüklenecek programı/kütüphaneyi değiştirir	1.1
Açık dosya tanıtıcıları	Ebeveynin gizli dosyası sızar; 0–2 kapalıysa ilk dosya "stdout" olur	1.5
<code>umask</code>	<code>000</code> → herkese yazılabilir dosyalar	2.7
Kaynak sınırları	Çekirdek dökümü açık → bellek diske	1.9
Kimlik ve yetkiler	setuid: gereğinden fazla yetki	1.3

Kural: kendi kurmadığın hiçbir şeye güvenme

Adım 1–2: Ortam ve tanıtıcılar

```
/* Ortam: kara liste değil BEYAZ liste */
clearenv();                               /* önce korunacakları kopyala! */
setenv("PATH", "/usr/bin:/bin", 1);

/* 0-2 kapalıysa /dev/null'a bağla, gerisini kapat */
for (int fd = 0; fd <= 2; fd++)
    if (fcntl(fd, F_GETFD) == -1 && open("/dev/null", O_RDWR) != fd) abort();
close_range(3, ~0U, 0);                   /* Linux 5.9+ */
```

- `getenv` işaretçisi `clearenv` sonrası **geçersiz**
- Kendi dosyalarınızda her zaman `O_CLOEXEC`

Adım 3–4: İzinler ve çökme dökümü

```
umask(077);                                /* rw----- */
int fd = open(yol, O_WRONLY | O_CREAT | O_EXCL, 0600);

struct rlimit r = { 0, 0 };
setrlimit(RLIMIT_CORE, &r);                /* döküm boyutu 0 */
prctl(PR_SET_DUMPABLE, 0, 0, 0, 0);        /* + ptrace ile bağlanılamaz */
```

- Windows: `SetErrorMode(SEM_FAILCRITICALERRORS | SEM_NOGPFAULTERRORBOX)`
- Asıl savunma: döküm alınsa bile içinde **sır kalmamalı**

Adım 5: Yetkiyi kalıcı bırak ve doğrula

```
if (setresgid(g, g, g) != 0) abort(); /* 1. önce GRUP */
if (setresuid(u, u, u) != 0) abort(); /* 2. sonra kullanıcı: r/e/s üçü de */
getresuid(&r, &e, &s);
if (r != u || e != u || s != u) abort(); /* 3. DOĞRULA */
```

- `seteuid()` geçicidir: saklı kimlik yetkili kalır (2. hafta Demo 13)
- Windows: **kısıtlanmış belirteç** (`CreateRestrictedToken`)
- Daha iyisi: **ayrıcalık ayrımı** — küçük yetkili süreç + büyük yetkisiz süreç (Tarif 1.4)

Adım 6: Program ve kütüphane yüklerken

❌ Hatalı	✅ Doğrusu
<code>system("convert " + dosya)</code>	<code>execve("/usr/bin/convert", argv, temiz_ortam)</code>
<code>execvp("convert", ...)</code>	Tam yolla <code>execve</code>
<code>CreateProcess(NULL, "C:\Program Files\...")</code>	<code>lpApplicationName</code> = tam yol, komut satırı tırnaklı
<code>LoadLibrary("yardimci.dll")</code>	<code>SetDefaultDllDirectories(LOAD_LIBRARY_SEARCH_DEFAULT_DIRS)</code> + tam yol

Windows DLL arama sırası = `PATH` sorununun ikizi (CWE-427)

Hepsi bir arada: `guvenli_baslat()`

```
int main(int argc, char **argv)
{
    tanitici_duzenle();    /* 1. tanıtıcılar */
    ortami_temizle();     /* 2. ortam */
    umask(077);           /* 3. izinler */
    dokumu_kapat();       /* 4. döküm */
    yetkiyi_birak();      /* 5. yetki */
    /* ... asıl iş ... */
}
```

Sıra önemli: başka hiçbir şey yapmadan önce

Demo 2 — Bellekte kalan parola (CWE-14)

```
char parola[64];
n = dosya_oku(fd, parola, sizeof(parola) - 1);
anahtar = anahtar_turet(parola, n);
memset(parola, 0, sizeof(parola));          /* sürüm 2 */
explicit_bzero(parola, sizeof(parola));      /* sürüm 3, Linux */
SecureZeroMemory(parola, sizeof(parola));    /* sürüm 3, Windows */
```

```
Hiç silmeyen   -> parola dökümde BULUNDU
memset (-02)   -> parola dökümde BULUNDU    ← !!!
explicit_bzero -> parola dökümde bulunamadı
```

Neden memset kayb oldu?

Derleyicinin düşüncesi:

" parola bundan sonra **hiç okunmuyor**. Ona sıfır yazmak sonucu değiştirmez. **Gereksiz — siliyorum.**"

Ölü yazma giderme (dead store elimination): hız için iyi, **sırlar için felaket**.

Platform	Kaldırılmayan silme
Linux (glibc)	<code>explicit_bzero</code>
Windows	<code>SecureZeroMemory</code>
C11 Ek K	<code>memset_s</code>
OpenSSL	<code>OPENSSL_cleanse</code>

+ `setrlimit(RLIMIT_CORE, 0)` · `mlock` · sırrı **kısa** tut, **kopyalama**

6. Süreç belleği ve taşmalar

Süreç belleği

yüksek adres

Yığın (stack) ↓	yerel değişkenler, dönüş adresleri
Öbek (heap) ↑	malloc
.data / .bss	küresel değişkenler
.text	makine kodu

düşük adres

C dizinin sonunu denetlemez. `ad[16]` 'ya 17. bayt → arkadaki değişkenin üzerine.

Yapının bellekteki hali

```
struct oturum { char ad[16]; int yonetici; };
```

```
offset:  0 ..... 15 | 16  17  18  19
         [ a  y  s  e \0 . . . . ] [ 00 00 00 00 ]  yonetici = 0

"AAAAAAAAAAAAAAAAAB" (17 karakter) kopyalanınca:
         [ A  A  A  A  A  A  A  A  A  A  A ] [ 42 00 00 00 ]  yonetici = 66 (!)
                                   'B'  '\0'
```

Küçük uçlu (little-endian): en düşük bayt en düşük adreste.

Arabellek taşması nedir?

Bir diziye **ayrılan boyuttan fazla** veri yazmak → taşan baytlar **komşu değişkenlerin** üzerine
C neden bu kadar kolay taşar?

1. Dizi **boyutunu bilmez**
2. Fonksiyona yalnız **başlangıç adresi** gider
3. Dizge = `'\0'` ile biten dizi; `strcpy` hedefi **sormaz**
4. Derleyici sınır denetimi **eklemez**

CWE-787 "sınırlar dışına yazma": CWE Top 25'in hep ilk sıralarında

Yığında bir taşma

```
void selamla(const char *ad) {  
    int yetkili = 0;  
    char tampon[16];  
    strcpy(tampon, ad);          /* ad > 15 karakter? */  
}
```

düşük adres	tampon[0..15]	<- yazma buradan başlar
	yetkili	<- biraz fazlası: MANTIK bozulur
	kaydedilmiş çerçeve	
yüksek adres	dönüş adresi	<- çok fazlası: ÇÖKME / akış ele geçer

Taşma türleri

Tür	Tipik neden	CWE
Yığın taşması	<code>strcpy</code> , <code>gets</code> , sınırsız <code>scanf("%s")</code>	121
Öbek taşması	Yanlış uzunluklu <code>memcpy</code>	122
Bir fazla (off-by-one)	<code><=</code> yerine <code><</code> ; <code>'\0'</code> için yer yok	193
Sınır dışı okuma	Karşı tarafın bildirdiği uzunluğa güvenmek	125
Tamsayı kaynaklı	<code>n * boyut</code> taşar → küçük blok, büyük kopya	190 → 787

Heartbleed (2014): hiçbir şey yazılmadı

- İstemci: "1 bayt gönderiyorum, **64 KB** diye bildiriyorum"
 - Sunucu: bildirilen uzunluğu **denetlemeden** 64 KB geri yolladı
 - Sızan: parolalar, oturumlar, hatta sunucunun **özel anahtarı**
 - CVE-2014-0160 · CWE-125 · Düzeltme: **tek bir uzunluk denetimi**
- ➔ Sınır dışı okuma da yazma kadar tehlikeli; üstelik **çökmez, iz bırakmaz**

Bir fazla hatası

```
char ad[8];  
for (int i = 0; i <= 8; i++)      /* 9 kez: ad[8] dışarıda */  
    ad[i] = kaynak[i];  
  
strncpy(ad, kaynak, sizeof ad); /* kaynak >= 8 → '\\0' YOK */  
printf("%s\\n", ad);             /* dizinin sonundan okumaya devam */
```

- `strncpy` güvenli `strcpy` **değildir**
- Tek bayt taşma bile komşu değişkenin en düşük baytını değiştirir

Tehlikeli fonksiyonlar → yerine

✗	✓
<code>gets(s)</code>	<code>fgets(s, sizeof s, stdin)</code> — <code>gets</code> C11'de kaldırıldı
<code>strcpy(d, s)</code>	Uzunluk denetimi + <code>memcpy</code> , <code>snprintf</code> , <code>strncpy</code> (glibc 2.38+)
<code>strcat(d, s)</code>	Kalan yeri hesapla; <code>snprintf</code> , <code>strlcat</code>
<code>sprintf(d, ...)</code>	<code>snprintf(d, sizeof d, ...)</code> + dönüş değeri denetimi
<code>scanf("%s", s)</code>	<code>scanf("%15s", s)</code>
<code>memcpy(d, s, n)</code>	Önce <code>n <= sizeof d</code>
<code>char a[n]</code> (VLA), <code>alloca</code>	Sabit üst sınır ya da <code>malloc</code>

Tarif 3.3–3.4 bugün: `snprintf` / `strncpy`, MSVC `strcpy_s`, C++ `std::string` / `span`

Doğru kalıp: önce doğrula, sonra sınırlı kopyala

```
int selamla(const char *ad)
{
    char tampon[16];
    size_t n = strlen(ad, sizeof tampon);    /* en fazla 16 bayt oku */
    if (n == sizeof tampon)                  /* uzunsa REDDET */
        return -1;
    memcpy(tampon, ad, n);
    tampon[n] = '\\0';                       /* sonlandırıcı her zaman */
    printf("Merhaba %s\\n", tampon);
    return 0;
}
```

Neden kesmek yerine reddetmek? `/home/ayse/gizli_rapor.txt` → `/home/ayse/gizli`

C++: boyutu tür taşır — ama dikkat

```
void selamla(const std::string &ad) {  
    if (ad.size() > 15) throw std::invalid_argument("ad cok uzun");  
    std::cout << "Merhaba " << ad << '\n';  
}
```

- `vector::operator[]` sınır **denetlemez** → `at()` denetler
- `c_str()` ile alınan işaretçiye yazmak → C'nin bütün sorunları geri gelir

Taşmayı kim yakalar?

Katman	Araç	Ne zaman?
Kod	Sınır denetimi, güvenli API	Her zaman
Uyarı	<code>-Wall -Wextra -Wformat-security · /W4 /sdl</code>	Geliştirme
Statik analiz	<code>clang --analyze , cppcheck , /analyze</code>	CI
Sanitizer	<code>-fsanitize=address · /fsanitize=address</code>	Test
Fuzzing	libFuzzer, AFL++	Test (4. hafta)
Derleyici	Kanarya <code>-fstack-protector-strong · /GS , _FORTIFY_SOURCE</code>	Sürüm
OS / donanım	DEP/NX, ASLR · Intel CET, ARM PAC	Sürüm

⚠ Koruma sömürüyü **zorlaştırır**, hatayı **gidermez** — kanarya programı sonlandırır = hizmet engelleme

ASan raporunu okumak

```
ERROR: AddressSanitizer: stack-buffer-overflow
WRITE of size 21 at 0x7ffd... thread T0
    #0 in strcpy
    #1 in selamla  ornek.c:7
    #2 in main     ornek.c:15
This frame has 1 object(s):
    [32, 48) 'tampon' <== Memory access at offset 48 overflows this variable
```

1. **Ne?** yığın taşması, 21 bayt yazma
2. **Nerede?** `ornek.c:7`, `strcpy`
3. **Hangi değişken?** `tampon` (16 bayt), erişim tam sınırın ötesinde

Yaklaşık 2× yavaş → yalnız **test** derlemesinde

Demo 3 — Taşmayla yetki yükseltme (CWE-121)

`.\demo.ps1` (Windows) · `sh demo.sh` (WSL / Linux)

Sürüm	17 karakterlik saldırı
<code>giris</code> (korumasız)	✗ YÖNETİCİ erişimi verildi
<code>giris_asan</code> (ASan)	✗ sessiz kaldı — taşma yapının içinde
<code>giris_asan</code> + 40 karakter	✓ stack-buffer-overflow yakalandı
<code>giris_denetimli</code> (<code>_FORTIFY_SOURCE=2</code> / <code>strcpy_s</code>)	✓ program durduruldu
<code>giris_guvenli</code>	✓ Reddedildi

Düzeltilme: doğrula, sonra sınırlı kopyala

```
/* İzin listesi: 1-15 karakter, harf/rakam/_/- */
static int ad_gecerli_mi(const char *s)
{
    size_t n = strlen(s, 16);
    if (n == 0 || n >= 16) return 0;
    for (size_t i = 0; i < n; i++) {
        unsigned char c = (unsigned char)s[i];
        if (!isalnum(c) && c != '_' && c != '-') return 0;
    }
    return 1;
}

struct oturum o = { .yonetici = 0 }; /* güvenli varsayılan */
snprintf(o.ad, sizeof(o.ad), "%s", argv[1]); /* boyutu bilen kopya */
```

Canary, NX, ASLR (4. hafta) **sömürmeyi zorlaştırır, hatayı düzeltmez.**

Tamsayılar: -1 ne zaman dev olur?

`int` işaretli · `size_t` işaretsiz · negatifler **ikiye tümleyen**

```
int    -1    = 0xFFFFFFFFFFFFFFFF
size_t      = 18 446 744 073 709 551 615
```

```
if (uzunluk > 16) return -1; /* -1 bu denetimi GEÇER */
memcpy(hedef, kaynak, uzunluk); /* int -> size_t */
```

Demo 4 — İşaretli uzunluk (CWE-195)

```
ADIM 0  GCC -Wall -Wextra: sessiz · -Wsign-conversion: uyarı  
        MSVC (C): sessiz · aynı kod C++ olarak: C4365  
ADIM 1  kopya 8      -> Kopyalandı: ORNEK-KA  
ADIM 2  kopya 100    -> Reddedildi  
ADIM 3  kopya -1     -> Linux: SIGSEGV (139) · Windows: 0xC0000409  
ADIM 4  ASan         -> negative-size-param: (size=-1)  
ADIM 5  kopya_guvenli -1 / 12abc -> Reddedildi
```

Kural: boyutlar `size_t` · alt ve üst sınır · `strtol` · uyarıları aç

7. Bellek yönetimi, bölümlleme ve güvenli süreç

Dinamik belleğin yaşam döngüsü

Ayrır → doğrula → kullan → sırrı sil → bir kez serbest bırak → işaretçiyi unut

Adım	Unutulursa	CWE
Doğrula	<code>malloc</code> NULL döner, denetlenmez → çökme	476
Kullan	Sınır dışı yazma	787
Sırrı sil	Blok bir sonraki <code>malloc</code> 'la başka koda gider	226
Bir kez bırak	Çift serbest bırakma	415
İşaretçiyi unut	Serbest bırakıldıktan sonra kullanma (UAF)	416
Hiç bırakmamak	Sızıntı → hizmet engelleme	401

UAF: neden bu kadar tehlikeli?

```
Kullanici *aktif = malloc(sizeof *aktif);
Kullanici *onbellek = aktif;           /* ikinci sahip */
free(aktif); aktif = NULL;             /* onbellek hâlâ eski adreste */

char *not = malloc(sizeof(Kullanici)); /* aynı blok geri verilebilir */
strcpy(not, "....");
if (onbellek->yetki) { ... }           /* 'not'un baytlarını okuyor */
```

- `free` belleği OS'e vermez, **boş listeye** koyar → aynı blok yeniden verilir
-  **Sahiplik kuralı:** her bloğun **tek sahibi** var, yalnız o serbest bırakır

Düzeltilmiş kalıplar

```
Kayit *k = calloc(n, sizeof(Kayit));          /* çarpım taşmasını denetler + sıfırlar */
if (k == NULL) return HATA_BELLEK;

Kayit *yeni = reallocarray(k, n2, sizeof(Kayit));
if (yeni == NULL) { free(k); return HATA_BELLEK; } /* k'yi kaybetme */
k = yeni;
```

✗ `tampon = realloc(tampon, n);` → başarısızsa eski blok kaybolur

✗ Sır tutan tamponda `realloc` → taşınan eski blok **sıfırlanmaz**

Sırları silmek (Tarif 13.2)

Platform	Fonksiyon
Linux, BSD	<code>explicit_bzero(p, n)</code>
Windows	<code>SecureZeroMemory(p, n)</code>
C11 Ek K / C23	<code>memset_s</code> / <code>memset_explicit</code>
Hiçbiri	<code>volatile</code> işaretçiyle bayt bayt

Sırrın **gizli kopyaları**: değerle geçirilen yapılar · `realloc` 'un eski bloğu · `std::string` (SSO, silmeden yok olur) · G/Ç tamponları · takas ve hazırda bekletme dosyası

Diske düşmesin (Tarif 13.3) ve RAIİ

```
mlock(anahtar, sizeof anahtar);          /* Windows: VirtualLock */  
/* ... kullan ... */  
cen429_sil(anahtar, sizeof anahtar);  
munlock(anahtar, sizeof anahtar);
```

```
class GizliAnahtar {                      /* yıkıcı otomatik siler */  
    ~GizliAnahtar() { cen429_sil(v_.data(), v_.size()); }  
    GizliAnahtar(const GizliAnahtar&) = delete; /* kopya = ikinci sır */  
};
```

Kilitleme **sayfa** düzeyinde, miktar sınırlı → yalnız küçük anahtar bölgesi

Bellek hatası avcıları

Araç	Bulduğu	Kullanım
AddressSanitizer	Taşma, UAF, çift bırakma	<code>-fsanitize=address</code> · <code>/fsanitize=address</code>
LeakSanitizer	Sızıntı	ASan ile (Linux)
MemorySanitizer	İklendirilmemiş okuma	Clang <code>-fsanitize=memory</code>
Valgrind	Hepsi (yavaş)	<code>valgrind --leak-check=full ./prog</code>
CRT hata ayıklama öbeği	Sızıntı	MSVC <code>_CrtSetDbgFlag</code>

Değerlendirici: işlem bittikten sonra döküm alır, test parolasını arar

Korunan kod bölümlleme: TCB'yi küçült

Güvenilir hesaplama tabanı (TCB): "hatalıysa güvenlik çöker" dediğimiz kodun tamamı

- 200.000 satırlık uygulama, 2.000 satırlık **korunan çekirdek**
- İnceleme, gizleme, bütünlük denetimi yalnız çekirdeğe → maliyet de yalnız orada

[Arayüz] → [İş mantığı] —yalnız tutamaç + şifreli veri—► [Korunan çekirdek]
kripto · anahtar · RASP

Bölümleme seçenekleri

Seçenek	Yalıtım	Örnek
Aynı süreçte ayrı modül	Düşük	Java uygulamasında native kütüphane
Ayrı süreç	Orta	Tarayıcı kum havuzu, yetkili yardımcı süreç (Tarif 1.4)
OS hizmeti	Orta–yüksek	DPAPI, Anahtarlık, Android Keystore
Donanım	Yüksek	Güvenli eleman, TPM, TEE
Sunucuya taşıma	En yüksek	Sır hiç istemciye gitmez, HSM'de kalır

En güçlüsü **sırrı istemciden kaldırmak**; olmazsa donanım; o da yoksa **yazılım koruması** (bu dersin konusu)

Arayüz: dar, doğrulanmış, sırsız

```
typedef struct KasaAnahtari KasaAnahtari;    /* opak tür */

int kasa_ac(const char *parola, size_t n, const unsigned char tuz[16],
            KasaAnahtari **cikti);           /* parolayı siler */
int kasa_coz(KasaAnahtari *a, const unsigned char *sifreli, size_t n,
            unsigned char *acik, size_t kapasite, size_t *yazilan);
void kasa_kapat(KasaAnahtari *a);           /* anahtarı siler */
```

1. **Sırrı dışarı verme, iş yaptır** (tutamaç)
2. Sınırı geçen her şeyi **doğrula**
3. Sınırdan **açık veri** geçirme

Kâhin ve kod kaldırma

- **Şifre çözme kâhini:** saldırgan `kasa_coz` 'u kendi verisiyle çağırır — anahtara gerek yok
- **Kod kaldırma:** korunan fonksiyonu olduğu gibi kopyalayıp kara kutu gibi çalıştırır

Önlem	Fikir
Cihaza/örneğe bağlama	Başka ortamda doğru sonuç çıkmaz
Sürüme bağlama	Eski/değiştirilmiş sürümde anahtar çözülmez
Bağlam denetimi	Çağıranın bütünlüğü ve akış sırası doğrulanır
Ortak statik anahtar yok	Bir kopyanın kırılması diğerlerini etkilemez

Şifreleme ile güvenli işleme: açıklık penceresi

Kötü: [aç]  [kapat] anahtar hep açık
İyi: [aç]  [kapat] yalnız işlem anında

1. **Tam zamanında çözme** → fonksiyon dönmeden sil
2. **Örtme**: anahtar XOR rastgele maske — bellekte iki anlamsız parça
3. **Parçalama**: farklı yerlerde, kullanımda birleştir
4. **Kod şifreleme**: fonksiyon kodu çalışma anında çözülür (4, 9. hafta)
5. **Whitebox**: anahtar hiç açılmaz (11. hafta)

Güvenli geliştirme yaşam döngüsü

Aşama	Etkinlik	Hafta
Gereksinim	Standartlardan güvenlik gereksinimi	13
Tasarım	Tehdit modeli, koruma planı	1–2
Gerçekleştirme	Kodlama kuralları, yasaklı fonksiyonlar, statik analiz	4–5
Doğrulama	Sanitizer, fuzzing, sızma testi	4, 12
Yayın	İmzalı derleme, benzersiz sürüm kimliği	1, 10
Yanıt	Zafiyet bildirimi, acil güncelleme	12

Microsoft SDL · NIST SSDF (SP 800-218)

Benzersiz sürüm kimliği

İncelenen ile dağıtılan aynı mı? → 1.4.2 + git kimliği + ikili dosyanın SHA-256'sı

```
execute_process(COMMAND git rev-parse --short=12 HEAD  
                OUTPUT_VARIABLE GIT_KIMLIK OUTPUT_STRIP_TRAILING_WHITESPACE)  
target_compile_definitions(kasa PRIVATE SURUM="1.4.2" GIT_KIMLIK="${GIT_KIMLIK}")
```

```
Get-FileHash .\kasa.exe -Algorithm SHA256 · sha256sum ./kasa
```

Sahada sürüm, **anahtar türetmeye** de katılır → sürüm geri alma saldırısı çalışmaz

Değişiklik yönetimi: yedi adım

1. Temel çizgi → 2. Değişiklik talebi → 3. Sınıflandırma → 4. Onay ve planlama → 5. Geliştirme ve test → 6. Yayın → 7. Doğrulama
- 3 ile 4 arasında **güvenlik etki analizi**: hangi varlık, arayüz, tehdit, önlem?
 - Yalnız değişen kısmın incelenmesi = **delta değerlendirme** (13. hafta)
 - Depo: korumalı dallar · en az bir gözden geçiren · imzalı etiket · MFA

Ödünleşim kaydı

Karar	Seçilen	Gerekçe	Kalan risk
Sürümde günlükleme	Derleme makrosuyla tamamen kaldırıldı	Susturulmuş kod dizge olarak kalır	Sahada tanı zor
Hata kodları	Yalnız başarılı/başarısız (opak)	Ayrıntı saldırgana ipucu	Ayrı tanı kanalı
Native "debuggable" bayrağı	Açık (gerekçeli istisna)	Bütünlük denetimi kendi belleğini okumalı	Debugger algılama ile telafi
Parola türetme süresi	250 ms	Bekleme ↔ kaba kuvvet dengesi	Çok zayıf parola

Yazılmamış ödünleşim = değerlendiricinin gözünde **hata**

Bu haftanın araç kutusu

İş	Linux / GCC	Windows / MSVC
Şüpheli kod uyarıları	<code>-Wall -Wextra -Wsign-conversion</code>	<code>/W4</code>
Kütüphanede boyut denetimi	<code>-D_FORTIFY_SOURCE=2 -O2</code>	<code>strcpy_s</code> gibi <code>_s</code> işlevleri
Bellek hatasını yakalama	<code>-fsanitize=address</code>	<code>/fsanitize=address</code>
Bellek dökümü	<code>gdb + gcore</code>	<code>MiniDumpWriteDump</code>
Makine kodu	<code>objdump -d</code>	VS Ayrıştırılmış Kod
Kaldırılmayan silme	<code>explicit_bzero</code>	<code>SecureZeroMemory</code>
Güvenle program çalıştırma	<code>posix_spawn</code> + mutlak yol	<code>CreateProcessW</code> + tam yol

8. Proje ve kapanış

Proje: bu hafta yapılacaklar

1. **Takım** (bireysel ya da en çok 4 kişi) ve **konu**
2. GitHub deposu + **proje planı** (iş paketleri, takvim) → onaylat
3. Güvenlik kılavuzu taslağı:
 - **S0** kapak ve sürüm geçmişi · **S2** ürün genel bakışı
 - **S3** mimari ve **arayüz tablosu**
 - **S4** saldırgan modeli · **STRIDE** · **saldırı ağacı**

Kendi başına çalış (not verilmez)

1. PATH tuzağı: `ls` (Linux) / `whoami` (Windows) ile tekrarla, tam yolla düzelt
2. Demo 2'de `MOD optimize` → `MOD korumasız`: `memset` şimdi çalışıyor mu?
3. Demo 3'te 16, 17, 18, 19, 20 karakter: `yonetici` değerleri tablosu
4. `struct oturum` 'a `bakiye` ekle: taşmayla 1000'in üstüne çıkar, sonra engelle
5. Kendi projenin STRIDE tablosu (≥ 8 tehdit + önlem)
6. Heartbleed raporunu oku: hangi denetim eksikti? Hangi CWE?

Ayrıntı ve ipuçları: ders sitesi → Hafta 1 → "Kendi başına çalış"

Kendini sına

1. Varlık, tehdit, zafiyet farkı?
2. Alıcı IBAN'ın yolda değişmesi CIA'nın hangisi?
3. Beyaz kutu saldırganı neyi farklı yapar?
4. STRIDE "E" hangi demo?
5. VE / VEYA düğümü — savunma için hangisi iyi?
6. `system("date")` neden tehlikeli? Üç düzeltme?
7. `memset` neden kayboldu?
8. ASan 17 karakteri neden yakalamadı?
9. `-1` `memcpy` 'ye gidince ne olur?
10. Derinlemesine savunma — mobil ödeme mimarisinden örnek?

Kendini sına — cevaplar (1–10)

1. **Varlık** korunacak değerli şey; **tehdit** zarar verebilecek olası olay/aktör; **zafiyet** tehdidin kullanabileceği kusur.
2. **Bütünlük (Integrity)** — yoldaki IBAN'ın değişmesi veri bütünlüğü ihlalidir.
3. **Beyaz kutu (MATE)** programa **sahiptir** (belleği/anahtarı okur, kodu değiştirir); ağ saldırganı yalnız **dışarıdan** bağlanır.
4. **E = Elevation of Privilege** (yetki yükseltme) → ayrıcalık bırakma / `setuid` demosu.
5. **VE (AND)** savunmaya iyi: hedef tüm alt adımları ister, **birini** kesmek saldırıyı durdurur. VEYA saldırgana kolaylık.
6. `system` **kabuğu** yorumlar → komut enjeksiyonu/PATH ele geçirme. Düzelt: `execvp` ile **doğrudan** çağır (kabuksuz), **tam yol**, girdi allowlist.
7. Derleyici **ölü mağaza** olarak sildi (sonra okunmuyor). `memset_s` / `explicit_bzero` / `volatile` kullan.
8. Taşma **redzone**'a ulaşmadı / o kod yolu çalışmadı; ASan yalnız **enstrümante** bölgeyi görür.
9. `-1`, `size_t` 'te **SIZE_MAX**'e döner → devasa kopya → taşma/çökme.
10. Ödeme: TLS+pinning (aktarım) · AEAD (bekleme) · RASP/anti-debug · whitebox/HSM anahtar · sunucu denetimi — **katman**.

Kendini sinama (devam)

1. Bir veri akışına neden **S** sorulmaz?
2. Tehdide dört yanıt nedir? "Ortadan kaldırma"ya örnek verin
3. setuid programda neden **önce grup** bırakılır?
4. `strncpy` neden güvenli `strcpy` değildir?
5. Heartbleed neden hiçbir şey çökmeden veri sızdırdı?

Kendini sına — cevaplar (devam 1–5)

1. Veri akışına **S sorulmaz**: akışın kimliği yoktur (sahtecilik uçlara/varlıklara sorulur).
2. **Azalt · kaldır · aktar · kabul et**. "Kaldırma" örneği: "parolayı hatırla" özelliğini **hiç sunmamak**.
3. `setuid` 'de **önce grup** bırakılır; kullanıcı önce bırakılırsa grubu değiştirecek **yetki kalmaz**.
4. `strncpy` uzun kaynakta sonuna sonlandırıcı **koymaz** → sonlandırılmamış dize.
5. Heartbleed yalnız **okudu**; okunan bellek sürecin kendi **geçerli** belleği — → çökmeden sızdı.

Kendini sinama (devam)

6. Kanarya taşmayı fark edince ne yapar? Neden hâlâ sorun?
7. `free(p); p = NULL;` neyi önler, neyi önlemez?
8. `mlock` neyi önler, neyi önlemez?
9. "Şifre çözme kâhini" nedir? Bir karşı önlem?
10. Benzersiz sürüm kimliğinin üç bileşeni?

Kendini sinama — cevaplar (devam 6–10)

6. Kanarya bozulunca programı **sonlandırır** (hizmet reddi); bitişik değişkene taşmayı **görmez**.
7. `free(p); p=NULL;` **çift bırakma/UAF**'ı önler; **takma adları** (aynı adrese başka işaretçi) korumaz.
8. `mlock` belleğin **takasa** yazılmasını önler; **okunmasını** ve silinmemiş sırrı önlemez.
9. **Şifre çözme kâhini**: anahtarsız çözdürme imkânı. Karşı önlem: bağlam/bütünlük denetimi, çağrı sınırı, **cihaza bağlama**.
10. Benzersiz sürüm: **anlamsal sürüm + git kimliği + ikili SHA-256**.

Gelecek hafta

Hafta 2 — Bilgisayar Virüsleri ve Güvenlik Modelleri

Kötü yazılım türleri · Bell–LaPadula, Biba, Clark–Wilson · CWE, OWASP, CVSS

Kaynak: Viega & Messier, Tarif 1, 3, 12.1, 13.2–13.3

Ek · STRIDE'ı adım adım uygulamak

Örnek sistem

Bir mobil bankacılık uygulaması:

- kullanıcı girişi
- bakiye görüntüleme
- para transferi

Her STRIDE harfini bu sisteme uygulayalım.

S · Spoofing (sahtecilik)

- **Tehdit:** başkası gibi davranmak (sahte kullanıcı/sunucu).
- **Örnek:** sahte sunucu, çalınan oturum.
- **Önlem:** güçlü kimlik doğrulama, TLS + sertifika denetimi.

T · Tampering (kurcalama)

- **Tehdit:** veriyi/kodu izinsiz değiştirmek.
- **Örnek:** transfer tutarını değiştirmek.
- **Önlem:** bütünlük (MAC/imza), bütünlük denetimi (RASP).

R · Repudiation (inkâr)

- **Tehdit:** yaptığıııı inkâr etmek.
- **Örnek:** "ben o transferi yapmadım".
- **Önlem:** güvenli günlük, imza, denetim izi.

I · Information disclosure (bilgi sızması)

- **Tehdit:** gizli verinin sızması.
- **Örnek:** bakiye/parola sızıntısı.
- **Önlem:** şifreleme, en az yetki, sızıntısız hata iletisi.

D · Denial of service (hizmet reddi)

- **Tehdit:** hizmeti kullanılamaz kılmak.
- **Örnek:** aşırı istek, kaynak tüketimi.
- **Önlem:** hız sınırı, kaynak kotaları, girdi sınırı.

E · Elevation of privilege (yetki yükseltme)

- **Tehdit:** izinsiz yetki kazanmak.
- **Örnek:** normal kullanıcı yönetici olur.
- **Önlem:** en az yetki, yetki denetimi, güvenli varsayılan.

STRIDE · özet uygulama

Harf	Bu sistemde önlem
S	TLS + kimlik doğrulama
T	MAC/imza + RASP
R	Güvenli günlük
I	Şifreleme + en az yetki
D	Hız sınırı
E	Yetki denetimi

Ek · Saldırı ağacı adım adım

Kök · hedef

HEDEF: Ödeme anahtarını ele geçir

Saldırganın nihai amacı. Şimdi yolları bölelim.

Dallar · nasıl?

Ödeme anahtarını ele geçir

- └ Bellekten oku (çalışırken)
- └ İkiliden çıkar (statik)
- └ Sunucudan sız (ağ)

Alt dallar · bellekten oku

Bellekten oku

- └─ Debugger bağla
- └─ Bellek dökümü al
- └─ Hook ile yakala

Her yaprak bir saldırı adımı.

Ağaçtan önceliklendirme

- En **kolay** yaprak en büyük risk.
- Önce onu kapat (ör. RASP + kısa ömür).
- Ağaç, savunma önceliğini gösterir.

Ağaçtan önleme eşleme

Yaprak	Önlem
Debugger	Anti-debug (6)
Bellek dökümü	Kısa ömür + koruma (6, 10)
İkiliden çıkar	Gizleme + whitebox (9, 11)
Ağdan sız	TLS + pin (10)

Ek · Güvenli tasarım ilkeleri derinleşme

En az yetki

- Her bileşen yalnız gerekeni yapabilsin.
- Fazla yetki = büyük saldırı yüzeyi.
- Örnek: DB kullanıcısı yalnız `SELECT` .

Güvenli varsayılan (fail-safe)

- Bir şey ters giderse **güvenli** tarafa düş.
- Hata → erişim **reddedilir** (fail-closed).
- Varsayılan: kapalı, kısıtlı.

Ekonomi ve açık tasarım

- **Ekonomi:** basit tasarım, az hata.
- **Açık tasarım:** güvenlik gizliliğe değil, **anahtara** dayanır (Kerckhoffs).
- Karmaşıklık düşmandır.

Tam aracılık ve ayırık ayrıcalık

- **Tam aracılık:** her erişim denetlenir (önbelleğe güvenip atlama).
- **Ayrık ayrıcalık:** kritik işlem için birden çok koşul.
- Derinlemesine savunmanın temeli.

İlkeler → bu ders

- Bu ilkeler dönem boyunca her haftada tekrar edecek.
- Kod, kriptoloji, RASP, gereksinimler — hepsi bunlara dayanır.
- İlkeler değişmez; teknikler değişir.

Ek · özet

- STRIDE her öğeye altı soru sorar.
- Saldırı ağacı hedefi yollara böler, önceliklendirir.
- Tasarım ilkeleri her kararın pusulası.

Tehdidi **modelle**, sonra **katmanla** koru.

Ek · Çözümlü kendini sinama

Soru 1

CIA üçlüsünü bir örnekle açıklayın.

Cevap: Gizlilik: parola yalnız sahibince görülür. Bütünlük: banka bakiyesi izinsiz değişmez.
Erişilebilirlik: uygulama gerektiğinde çalışır.

Soru 2

Hata ile zafiyet arasındaki fark nedir?

Cevap: Her zafiyet bir hatadır ama her hata zafiyet değildir. Zafiyet, bir saldırganın **kötüye kullanabileceği** hatadır.

Soru 3

MATE (beyaz kutu) saldırgan ağ saldırganından nasıl ayrılır?

Cevap: MATE programa sahiptir: kodu okur, belleği görür, değiştirir. Ağ saldırganı yalnız dışarıdan bağlanır.

Soru 4

STRIDE neyi sağlar?

Cevap: Tehditleri altı sınıfta sistemli aramayı: sahtecilik, kurcalama, inkâr, bilgi sızması, hizmet reddi, yetki yükseltme.

Soru 5

Saldırı ağacı ne işe yarar?

Cevap: Bir saldırı hedefini alt adımlara böler; en kolay/olası yolu ve önceliklendirmeyi gösterir.

Soru 6

Neden tek bir önlem yetmez?

Cevap: Her önlem aşılabılır; katmanlı savunmada biri aşılsa diğeri durdurur. Güç birliktelikten gelir.

Soru 7

"Gizleme doğru kodun yerine geçer mi?"

Cevap: Hayır. Gizleme okumayı zorlaştırır ama hatayı düzeltmez. Önce güvenli kod, sonra gizleme.

Soru 8

En az yetki ilkesi nedir?

Cevap: Her bileşen yalnız **gereken** yetkiye sahip olmalı; fazlası saldırı yüzeyini büyütür.

Soru 9

Bir güven sınırı nedir?

Cevap: Güvenilen ve güvenilmeyen bölgelerin ayrıldığı yer; girdi doğrulama en erken burada yapılır.

Soru 10

Kalan risk neden yazılır?

Cevap: Hiçbir sistem %100 güvenli değildir; bilinen ve kabul edilen riskler açıkça belgelenir.

Ek · İşlenmiş mini tehdit modeli

Ürün · sentetik

Bir "parola kasası" uygulaması:

- parolaları yerelde saklar
- ana parolayla açılır

Hızlı bir tehdit modeli yapalım.

Adım · varlıklar

- Ana parola (C/I)
- Saklanan parolalar (C/I)
- Şifreleme anahtarı (C/I)

Adım · tehditler (STRIDE)

- **I** (bilgi sızması): cihaz çalınırsa parolalar okunur.
- **T** (kurcalama): kasa dosyası değiştirilir.
- **E** (yetki): kök cihazda belleğe erişim.

Adım · önlemler

- Parolalar AEAD ile şifreli ($I \rightarrow C$).
- Ana paroladan KDF ile anahtar.
- Bütünlük etiketi (T).
- Kısa ömürlü bellek + RASP (E).

Adım · kalan risk

- Köklü cihazda ana parola girilirken bellekte açık olabilir.
- Azaltma: hızlı silme, cihaz bağlama.
- **Açıkça yazılır.**

Sözlük

Terim	Anlam
CIA	Gizlilik/bütünlük/erişilebilirlik
MATE	Programa sahip saldırgan
STRIDE	Altı tehdit sınıfı
Saldırı ağacı	Hedefi alt adımlara bölme
DFD	Veri akış diyagramı
Katmanlı savunma	Çok önlem birlikte

Bu haftadan projeye

- **S2–S5:** ürün, mimari, varlık listesi, tehdit modeli.
- **S1:** kapsam, kaynaklar.
- Her varlık C/I/I+ ile etiketli; her tehdide bir önlem.

Son söz (1. hafta)

Güvenlik "his" değil, **süreçtir**: varlığı tanı, tehdidi modelle, katmanla koru, kalan riski yaz.

Bu çerçeveyi dönem boyunca kullanacağız.